MEAT 2.0

Competitive reverse engineering & developer upgrade brief | 17 October 2026

Executive direction

Build an original, adults-only social discovery platform combining proximity discovery, mutual matching, profile directories, events, private sharing and privacy-preserving location features. This is a clean-room specification, not a reproduction of proprietary code or assets.

Competitive reconstruction

Reference	Reference pattern	MEAT implementation
Grindr	Proximity grid, filters, temporary availability, private albums	Nearby discovery, intent status, revocable albums
Tinder	Swipe-based mutual matching, interest discovery, verification	Compatibility cards and consent-based matching
Manhunt	Profile directory and direct messaging (requires current-flow validation)	Directory, saved searches and profile browsing
RedHotPie	Couples/groups, events, clubs, community participation	Joint profiles, events, clubs and community
Sniffies	Map-first local discovery (specific controls require validation)	Coarse-position radar and expiring presence

Evidence boundary

Observed: high-level public-facing capabilities of Grindr, Tinder and RedHotPie. **Inferred:** current Manhunt and Sniffies interaction specifics, pending direct UI validation. **Unknown:** competitors' private algorithms, backend schemas and infrastructure. **Proposed:** all MEAT implementation choices below. Do not treat proposed technology as evidence of competitor internals.

Core development modules

DISCOVER: Nearby grid, preference filters, saved profiles and search.

RADAR: Map discovery with obfuscated location, no default exact coordinates.

MATCH: Mutual interest, intent filters, compatible introductions.

CONNECT: Real-time conversations, media permissions, reporting and blocking.

RIGHT NOW: Time-limited availability sessions with explicit opt-in.

VAULT: Private albums with revocation, access audit and expiry.

CIRCLES & SCENE: Couples/groups, communities, event listings and RSVPs.

MEAT AI & MEAT+: Opt-in assistance and premium entitlements without compromising safety.

Architecture proposal

Client: Next.js web/PWA and React Native mobile, shared design tokens. Backend: API gateway, identity and age assurance, discovery, matching, messaging, event and moderation services. Data: PostgreSQL with PostGIS for spatial queries, Redis for short-lived presence, encrypted object storage for approved media. Use scoped authorization, rate limits, abuse monitoring, data deletion and audit trails.

Key user journeys

1. Age assurance and account setup -> privacy choices -> profile creation -> discover. 2. Discover -> view profile -> express interest -> mutual match or permitted direct contact -> chat. 3. Opt into temporary presence -> approximate discovery -> expire automatically. 4. Share private album -> revoke permission -> recipient loses future access. 5. Report/block -> immediate interaction restriction -> moderation queue and appeal workflow.

Build It Better

MEAT MODE: Dating, Social, Exploring and Right Now with distinct visibility controls. **MEAT CHEMISTRY:** opt-in compatibility using intentions, boundaries and preferences. **MEAT PASSPORT:** temporary travel-city discovery with expiry and location safeguards.

Release gates and test matrix

Functional: account recovery, filters, mutual matching, message delivery, permissions, expiry and RSVP state. Security: authorization isolation, abuse throttling, media access revocation, safe uploads and deletion. Privacy: approximate-location disclosure, explicit consent, sensitive data minimisation and retention controls. Quality: mobile responsiveness, accessibility, offline/empty/error states, monitoring and load tests. Legal: Australian privacy, online safety, age assurance and applicable state-based requirements reviewed before beta.

Current build status

A standalone front-end developer prototype ZIP was prepared, containing synthetic demo interactions. It is not integrated with the existing MEAT source repository and is not production-ready. Real authentication, persistent backend, geolocation, moderation, payments and live messaging remain implementation tasks. Repository URL or source ZIP is required for a proper integration, code review and regression test run.

Implementation sequence

Phase 1: inspect current repository, map gaps, establish tests and design tokens. **Phase 2:** build identity, age assurance, profiles, discovery, matching and messaging. **Phase 3:** implement location privacy, private albums, reports and admin moderation. **Phase 4:** add community, events, subscriptions and opt-in AI. **Phase 5:** penetration testing, accessibility checks, legal review, beta rollout and telemetry.